SOFTWARE REQUIREMENTS SPECIFICATION

(SRS DOCUMENT)

for

Rail and Road-based Supply Chain Distribution System

Version 1.0

By

GROUP - 19

ATHISHAYA J.	240051V
JAYARATHNA M.G.P.S.	240267M
RANASINGHE R.G.A.D.B.	240548C
THANANCHEYAN E.	240645V

28-07-2026

Department of Computer Science and Engineering

University of Moratuwa

Contents

Revision History 3
1. Introduction 4
1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.4 Product Scope 4
1.5 References 4
2. Overall Description 5
2.1 Product Perspective 5
2.2 Product Functions 5
Order Management 5
Scheduling and Resource Management 5
Delivery Tracking 6
Reporting and Analytics 6
System Administration 6
User Communication 6
Data Flow Overview (Conceptual) 6
2.3 User Classes and Characteristics 6
2.4 Operating Environment 7
Hardware Platform 7
Software Platform 7
2.5 Design and Implementation Constraints 7
2.6 User Documentation 8
2.7 Assumptions and Dependencies 8
3. External Interface Requirements 8
3.1 User Interfaces 8
3.2 Hardware Interfaces 8
3.3 Software Interfaces 9
3.4 Communications Interfaces 9
4. System Features 9
4.1 Order Management 9
4.2 Train Scheduling 10
4.3 Truck Scheduling and Last-Mile Delivery 11
4.4 Driver and Assistant Rostering 12
4.5 Delivery Tracking 13
4.6 Report Generation 14
5. Database Design Overview 16

5.1 Design Principles 16
5.2 Logical Structure of Data 16
ER Diagram 20
5.3 Stored Procedures and Functions 21
5.4 Triggers 21
5.5 Indexing Strategy 21
5.6 Test Data 22
6. Other Nonfunctional Requirements 22
6.1 Performance Requirements 22
6.2 Security Requirements 23
6.3 Software Quality Attributes 23
6.4 Business Rules 23
7. Other Requirements 23
Appendix A: Glossary 24
Terms 24
Acronyms 24
Abbreviations 25
Appendix B: Analysis Models 25
Appendix C: To Be Determined List 25

Revision History

Version	Date	Description
1.0	28/07/2026	Initial document creation

1. Introduction

1.1 Purpose

This document specifies the software requirements for the **Rail and Road-based Supply Chain Distribution System (SCDS)** being developed for **Kandypack**, an FMCG manufacturer based in Kandy, Sri Lanka. It describes the functional and non-functional requirements, the data model and the business rules that the system must follow. This document serves as a guide for the design, implementation, testing and maintenance of the SCDS.

1.2 Document Conventions

- This SRS follows the IEEE Std 830-1998 standard with minor modifications according to the project requirements.
- **Bold text** is used to highlight important terms and terms defined in the Glossary (Appendix A).
- *Italic text* is used to indicate references to external documents or web resources.
- Text written inside **[square brackets]** represents optional or conditional information.
- Functional requirements are identified using the format **FR-<feature>. <number>** to make them easier to track.

1.3 Intended Audience and Reading Suggestions

Audience	Reason
Developers	Responsible for designing, implementing, testing, and maintaining the SCDS. They should read the complete document, especially Section 4 (System Features) and Section 5.2 (Logical Structure of Data).
Database implementers	Should focus on Section 5.2 (Entities, ER Diagram and Indexing) and Section 5.5 (Business Rules, Stored Procedures and Triggers).
Product Managers	Responsible for planning, coordinating and monitoring the development process. They should read Sections 1-3 and Section 4.
Testers / QA	Use the functional requirements in Section 4 and business rules in Section 5.5 to create test cases.

1.4 Product Scope

The system replaces Kandypack's existing Excel-based logistics tracking process with a centralized, database-driven platform that:

- Automates train cargo capacity scheduling, including transferring excess orders to the next available trips.
- Manages last-mile truck deliveries based on predefined routes.
- Schedules drivers and assistants while ensuring compliance with working-hour limits and assignment rules.
- Tracks orders and delivery statuses throughout the complete delivery process.
- Generates operational and sales reports to support management decisions.

1.5 References

- Sri Lanka Railways cargo logistics guidelines.

- IEEE Std 830-1998 Software Requirements Specification template.
- Project 3: Rail and Road-based Supply Chain Distribution System assignment brief.

2. Overall Description

2.1 Product Perspective

The SCDS is a new, self-contained system developed to replace Kandypack's existing Excel-based logistics tracking process. It:

- Replaces spreadsheets used for managing orders, delivery schedules and driver rosters.
- Automates logistics operations by enforcing capacity limits, scheduling rules, and delivery status updates.
- Provides reporting features to support management decisions.
- May support future integration with external services such as mapping or notification systems.

The system interacts with:

- **Sri Lanka Railways** - Provides train transportation information such as available trips and cargo capacity for deliveries between Kandy and the selected destination cities.
- **Kandypack staff and operations** - Manages order information, delivery schedules, and status updates.
- **Web browser** - Provides interfaces for customers, store managers, drivers, assistants and administrators.

2.2 Product Functions

Order Management

- Allows customers to place orders containing multiple products with quantities and a delivery address.
- Validates orders based on route coverage and the 7-day minimum advance-order requirement.
- Assigns order items to train trips based on available cargo capacity.
- Assigns order items to trucks for last-mile delivery after goods reach the destination store.

Scheduling and Resource Management

- Manages train schedules with automatic allocation of excess quantities to the next available trip.
- Manages truck schedules while checking conflicts between trucks, drivers, and assistants.
- Maintains driver and assistant rosters according to weekly working-hour limits (40 hours for drivers and 60 hours for assistants) and consecutive-assignment rules.
- Handles scheduling conflicts among available resources.

Delivery Tracking

- Tracks delivery status of each item from dispatch until completion.
- Sends delivery status updates to customers.
- Records delivery completion details.

Reporting and Analytics

- Generates quarterly sales reports based on value and volume.
- Provides reports on the most ordered items in each quarter.
- Generates city-wise and route-wise sales and delivery reports.
- Provides driver and assistant working-hour reports.
- Generates monthly truck usage analysis.
- Provides customer order history with delivery details.

System Administration

- Manages master data such as products, routes, trucks, drivers, and train schedules.
- Manages user roles and permissions.

User Communication

- Sends order status notifications to customers.
- Supports internal communication between dispatchers and store managers.

Data Flow Overview (Conceptual)

Customer order process:

> Customer → Order Management → Train Scheduling → Truck Scheduling → Delivery Confirmation → Order Status Update → Report Generation

Logistics operation process:

> Dispatcher → Train Scheduling → Truck Scheduling → Resource Conflict Checking → Delivery Tracking → Report Generation

2.3 User Classes and Characteristics

The main users of the SCDS are Customer, Dispatcher, Store Manager, Delivery Crew (Driver and Assistant) and System Admin. Each user has different responsibilities based on their role in the logistics process.

Actor	Primary Use Case	Description
Customer	Place Order	The system validates route coverage and checks the 7-day advance-order requirement before saving the order with "Pending" status.

Dispatcher	Schedule Train and Truck Deliveries	Assigns order items to train trips based on available cargo capacity. Any excess quantity is moved to the next available trip. Assigns trucks, drivers and assistants while following scheduling rules.
Store Manager	Confirm Reception and Dispatch Trucks	Confirms the arrival and unloading of goods, verifies assigned routes and confirms truck dispatch.
Driver / Assistant	Acknowledge Trips and Report Completion	Views assigned trips, confirms assigned deliveries and updates delivery completion status. The system records working hours and assignment details.
System Admin	Master Data, Schedules and Reports	Manages products, routes, trucks, train schedules, user roles and accesses system reports.

2.4 Operating Environment

Hardware Platform

- Development : Quad-core processor (Intel Core i5 / AMD Ryzen 5 or equivalent)
  8 GB RAM (16 GB recommended)
  10 GB or more available SSD storage
  Internet connection for package installation and deployment
- Production : Cloud hosting platform (Vercel, Railway, Neon PostgreSQL)

Software Platform

- Frontend : Vite.js, React 19, TypeScript, Tailwind CSS, Shadcn/ui
- Backend : Express.js, TypeScript, Prisma ORM, JWT Authentication
- Database : PostgreSQL 17
- Containerization: Docker 28, Docker Compose
- Hosting : Frontend - Vercel
  Backend - Railway
  Database - Neon PostgreSQL
- CI/CD : GitHub Actions

2.5 Design and Implementation Constraints

- Technology stack fixed as above (chosen by the team; not client mandated)
- Production hosting must support Docker based deployment with the minimum hardware/network stated in Section 2.4
- CI/CD must run against a Git based repository
- Each service must run reliably in an isolated container; services that cannot are out of scope
- Development machines require at least a quad core CPU and 8 GB RAM

- All business rule enforcement that affects data integrity (capacity limits, roster limits, no double booking) must be enforced at the database layer via constraints, triggers and stored procedures - not only in application code - so integrity holds regardless of which client writes the data (see Section 5.5)

2.6 User Documentation

- A user manual will be provided to explain the main system features and workflows for different user roles, including Admin, Dispatcher, Store Manager, Driver and Assistant.
- Basic guidance and walkthroughs may be provided to help users understand the system operations.

2.7 Assumptions and Dependencies

- Sri Lanka Railways train schedule information is entered manually and is assumed to be available and accurate.
- Route information is maintained manually by Kandypack staff.
- Products are assumed to be transported from Kandy to the six specified destination cities, and city-to-city transportation is not considered.
- The system is assumed to operate on a reliable hosting environment with sufficient resources.
- All users access the system through the web-based interface.
- Each destination city is assumed to have a designated store near the railway station for unloading and last-mile delivery operations.

3. External Interface Requirements

3.1 User Interfaces

- Clean, minimal, mobile friendly UI; header + left navigation + main content layout
- Standard buttons: Save, Update, Cancel, Delete, Submit, Back
- Confirmation/warning modals for destructive or conflict prone actions (e.g. “Are you sure you want to assign this route?”)
- Errors surfaced as red banners/toasts with a clear cause, e.g. “Driver is already assigned to a conflicting route.”
- Accessibility: tooltips on all buttons; tab based navigation
- Main UIs: (A) Admin/Dispatcher Dashboard
  (B) Driver/Assistant Interface
  (C) Customer facing responsive interface

3.2 Hardware Interfaces

The system interfaces with users only through their devices' web browsers; no direct hardware interface is required for v1.

3.3 Software Interfaces

Interface	Version	Reference
Frontend - React	19	react.dev
Build tool - Vite.js	7	vite.dev
Runtime - Node.js	22	nodejs.org
Backend - Express.js	5	expressjs.com
Database - PostgreSQL	17	postgresql.org
ORM - Prisma	6	prisma.io
Language - TypeScript	5.9	typescriptlang.org
Container - Docker	28	docker.com

3.4 Communications Interfaces

The system follows a three-tier architecture consisting of the client application, backend server, and database.

- Client ↔ Backend: HTTPS using RESTful APIs with JSON as the data exchange format.
- Backend ↔ Database: PostgreSQL protocol over TCP/IP using Prisma ORM.
- Client ↔ Backend Authentication: JSON Web Tokens (JWT) transmitted over HTTPS for secure authentication and authorization.

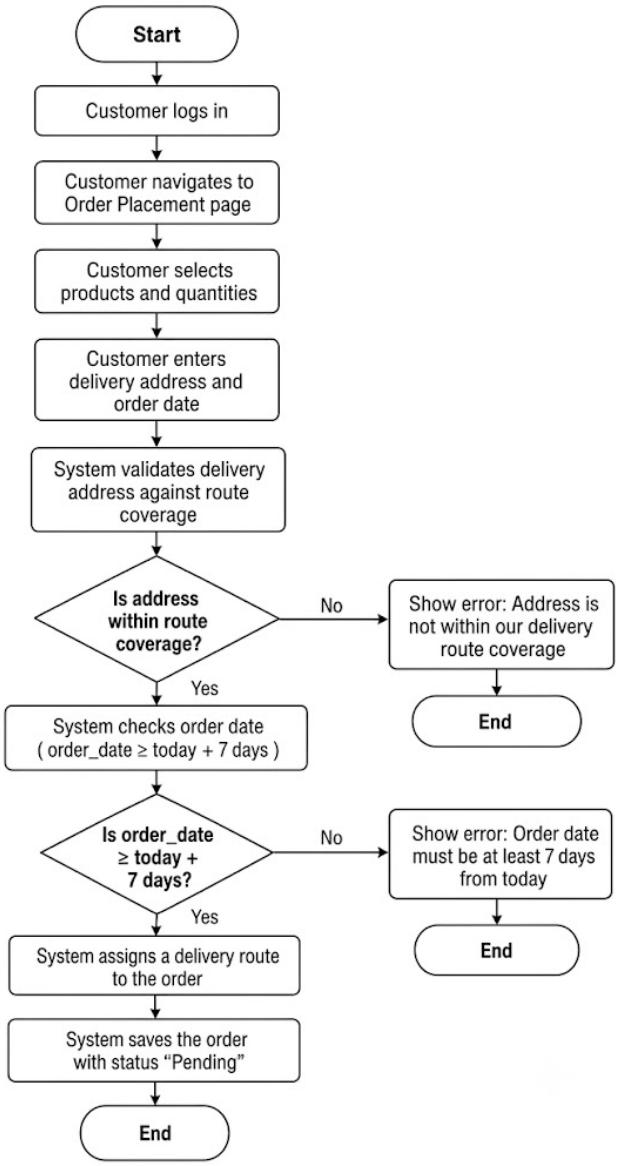
4. System Features

4.1 Order Management

4.1.1 Description and Priority

Customers place orders containing multiple products with quantities and a delivery address. Orders are validated for route coverage, delivery timing, and the 7-day advance-placement rule. High priority.

4.1.2 Stimulus/Response Sequence



4.1.3 Functional Requirements:

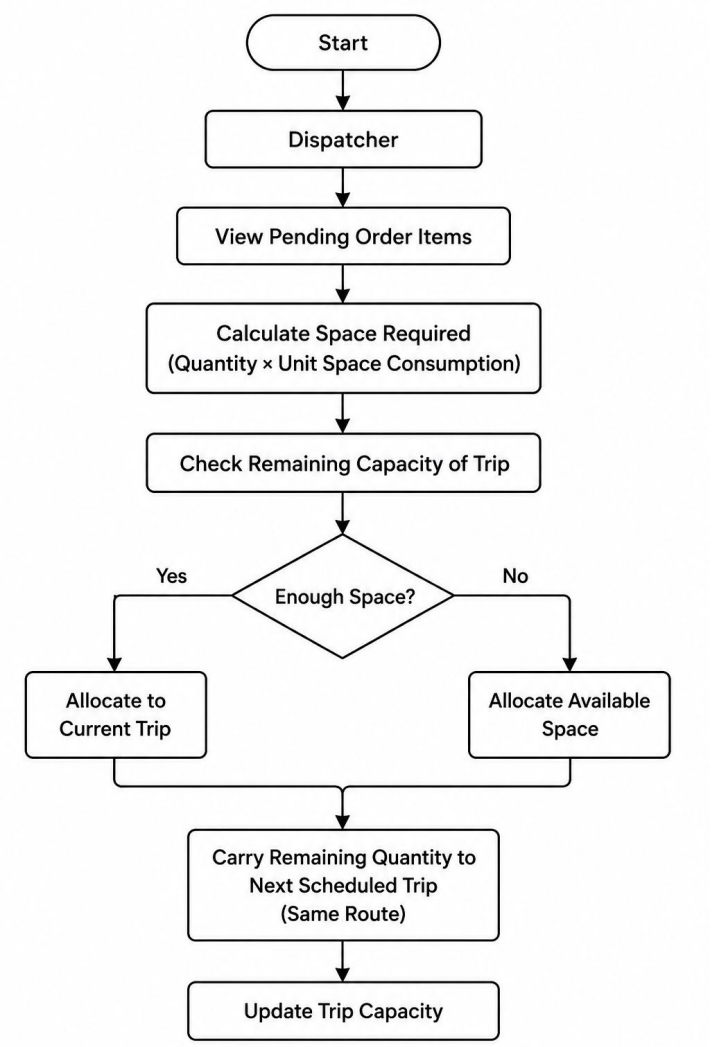
- The system shall reject an order whose delivery address does not map to an existing Route.
- The system shall reject an order placed less than 7 days before the requested delivery date.
- The system shall allow an order to contain one or more OrderItem rows, each referencing a Product and quantity
- The system shall persist new orders with status "Pending" and notify the customer on subsequent status changes.

4.2 Train Scheduling

4.2.1 Description and Priority:

The system assigns order items to train trips by cargo capacity and product space consumption; overflow moves to the next available trip. High priority.

4.2.2 Stimulus/Response Sequence:



4.2.3 Functional Requirements:

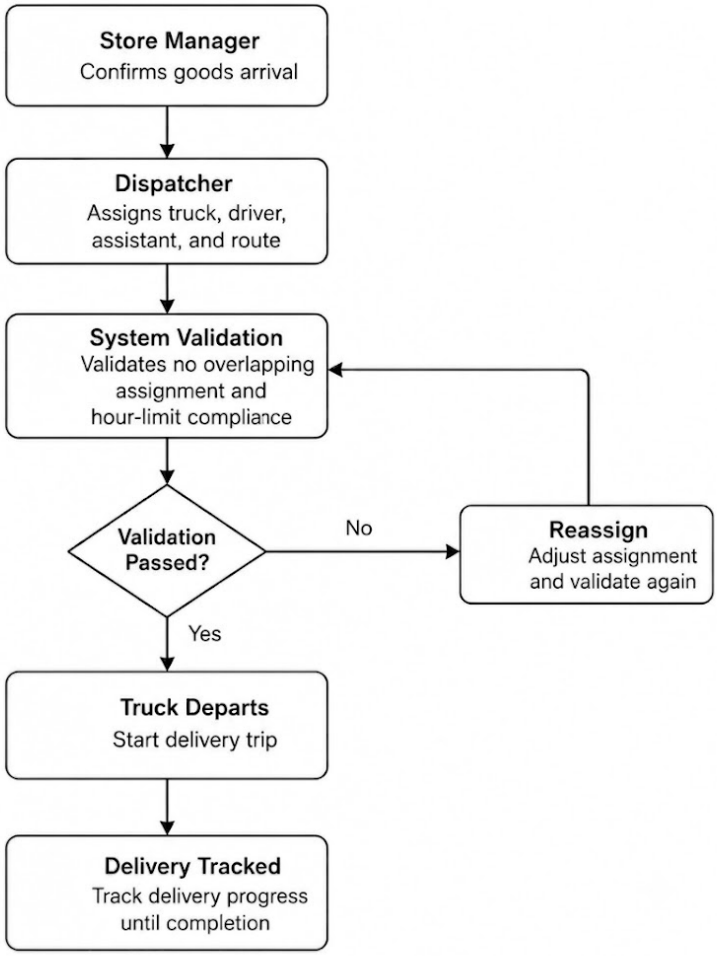
- The system shall compute required space for each OrderItem as quantity × Product.unit_space_consumption.
- The system shall never allocate more than a TrainTrip's max_capacity (enforced by a database trigger, not application logic alone - see Section 5.5).
- Where a single OrderItem cannot fit entirely within one trip's remaining capacity, the system shall split it across an OrderTrainAllocation row on the current trip and one or more rows on subsequent trips, preserving total quantity.
- The system shall run allocation inside a single database transaction so that a failure partway through never leaves an OrderItem partially, inconsistently allocated.

4.3 Truck Scheduling and Last-Mile Delivery

4.3.1 Description and Priority:

Order items arriving at a store are scheduled onto trucks for last-mile delivery; trucks, drivers and assistants must not be double-booked. High priority.

4.3.2 Stimulus/Response Sequence:



4.3.3 Functional Requirements:

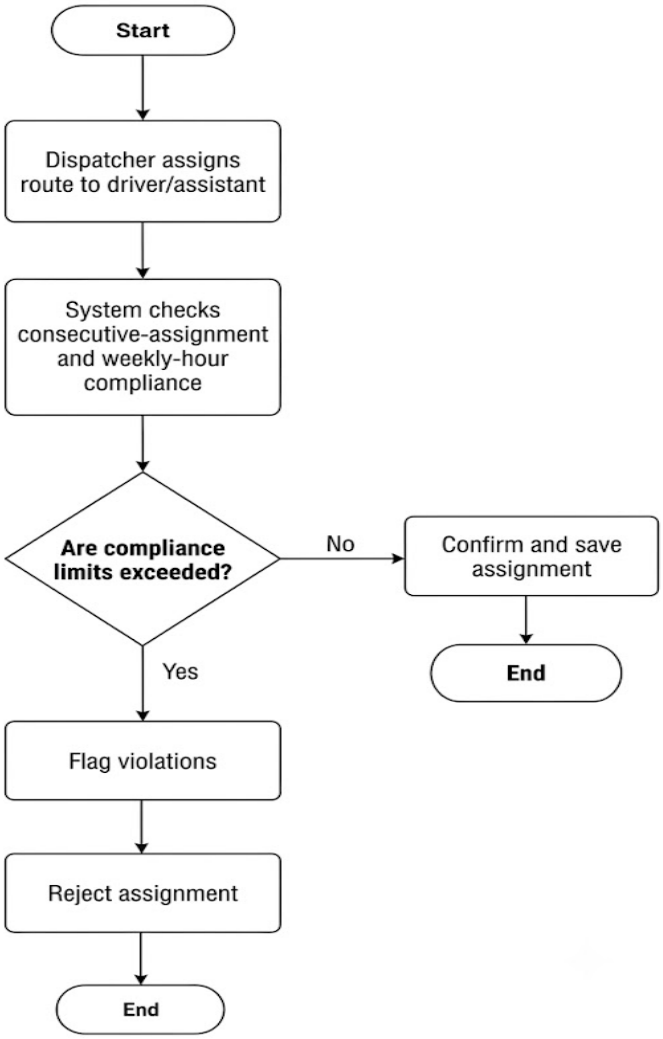
- The system shall assign each TruckTrip to exactly one predefined Route.
- The system shall reject a TruckTrip assignment if the truck, driver or assistant already has an overlapping time window on another TruckTrip.
- The system shall enforce driver/assistant weekly working-hour limits (Section 5.5) at assignment time, not only when a report is later generated.
- The system shall update per-item delivery status in real time as DeliveryItem rows change state.

4.4 Driver and Assistant Rostering

4.4.1 Description and Priority:

Ensures fair workload distribution and legal working-hour compliance. Medium priority.

4.4.2 Stimulus/Response Sequence:



4.4.3 Functional Requirements:

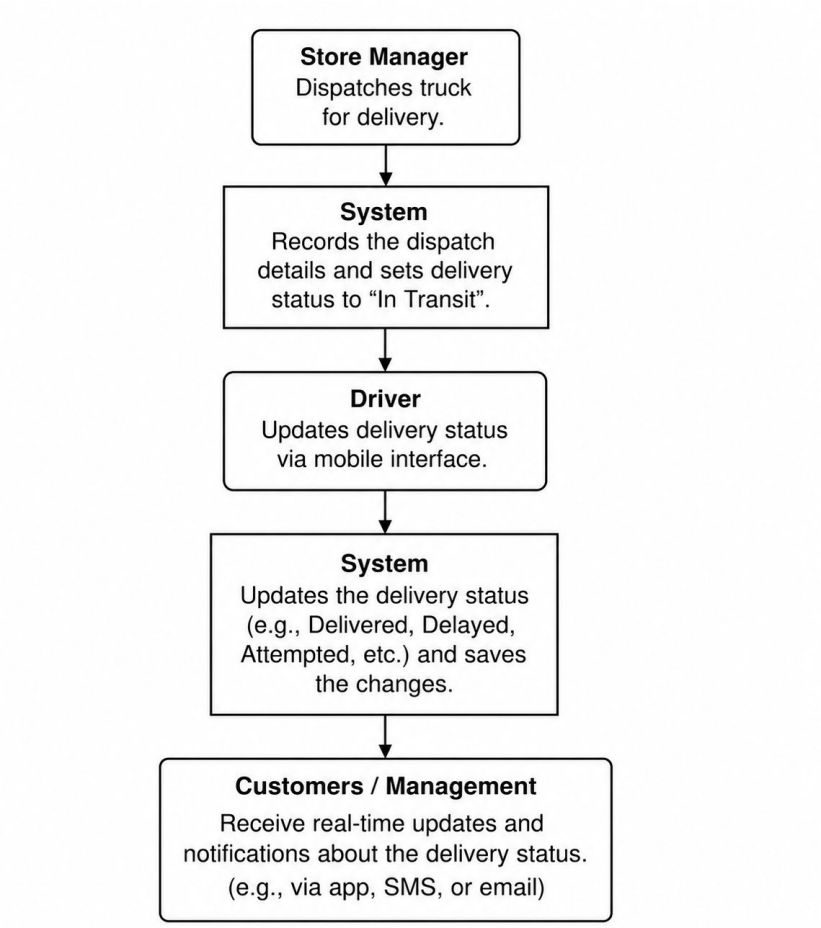
- A driver shall not be scheduled for two consecutive truck deliveries.
- An assistant shall not be scheduled for more than two consecutive routes.
- Weekly hours shall not exceed 40 for drivers or 60 for assistants; the check shall use a rolling ISO week computed from TruckTrip start/end times, not a self-reported total.

4.5 Delivery Tracking

4.5.1 Description and Priority:

Tracks order status from dispatch to delivery and notifies customers/management. Medium priority.

4.5.2 Stimulus/Response Sequence:



4.5.3 Functional Requirements

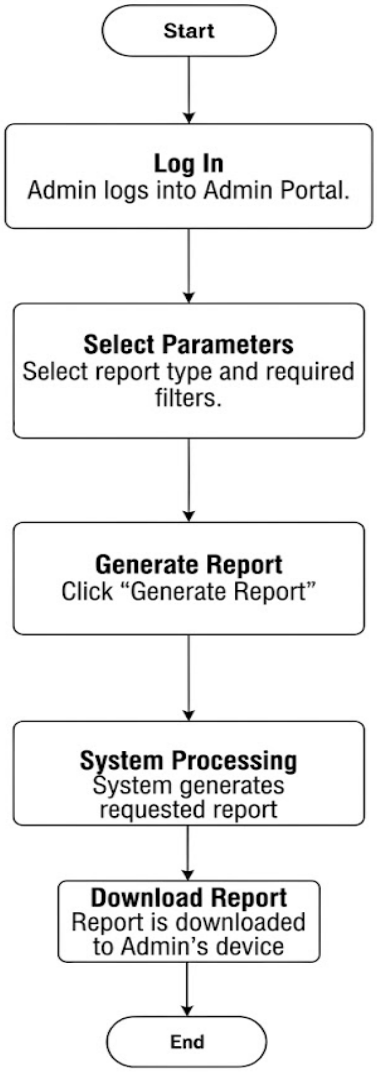
- Each DeliveryItem shall progress Pending → Scheduled → Delivered (or Failed/Returned).
- Status changes shall be timestamped and recorded, not overwritten.
- Customers shall be notified when all DeliveryItems for their order reach a terminal state.

4.6 Report Generation

4.6.1 Description and Priority:

Admins of the system, i.e. the management team at Kandypack, should be able to generate reports from predefined templates, in a timely manner. High priority

4.6.2 Stimulus/Response Sequences:



4.6.3 Functional Requirements:

- Quarterly sales report (value and volume).
- Most-ordered items per quarter.
- City-wise and route-wise sales/delivery breakdown.
- Driver/assistant working-hours report.
- Monthly truck-usage report.
- Customer order history with delivery details.
- Reports shall be exportable as PDF.

5. Database Design Overview

Because this project's main deliverable is a database rather than an application, this section sets out how the logical data model, the integrity rules and the reporting needs described earlier translate into concrete database objects: tables, keys, procedures, functions, triggers and indexes.

5.1 Design Principles

- Every entity has a single-column primary key and every relationship is enforced with a foreign key, so referential integrity is guaranteed by the database rather than by application code alone.
- An order can contain several products, so orders and order lines are modelled as two separate tables (Order and OrderItem) rather than repeating order details on every line, in line with normal form.

- Rules that must never be broken - cargo capacity, double-booking and working-hour limits - are enforced with triggers and stored procedures at the database level, so they hold even if a bug in the application layer would otherwise allow a bad write.

- Multi-step operations, such as placing an order with several lines or assigning a truck trip and updating several order statuses at once, are wrapped in stored procedures so they either fully succeed or fully roll back, preserving the database's atomicity and consistency (two of the ACID properties).

5.2 Logical Structure of Data

User entity

Property	Type	Description
id	Text	Primary key.
username	Text	Unique login name.
password_hash	Text	Salted hash of the password.
name	Text	Display name.
role	Text	Either “customer” or “worker”

Customer entity

Property	Type	Description
id	Text	Shares its primary key with the owning User row.
street_name	Text	Street name.
city	Text	Customer's city.
postal_code	Text	Postal code.
phone_no	Text	Contact number.

Worker entity

Property	Type	Description
id	Text	Shares its primary key with the owning User row.
type	Text	One of “driver”, “assistant” or “store_manager”
hourly_pay	Float	Pay rate per hour.

Store entity

Property	Type	Description
id	Text	Primary key.
name	Text	Store name.
city	Text	City the store is located in / serves as the rail unloading point for.
location	Text	Address or geo-reference.
managed_by	Text	Foreign key :The store_manager-type Worker responsible for this store.

Route entity

Property	Type	Description
id	Text	Primary key.
store_id	Text	The store this route is dispatched from. (FK→Store)
max_delivery_time	Interval	Maximum allowed delivery time, in hours.
area_name	Text	The delivery area this route covers.

Product entity

Property	Type	Description
id	Text	Primary key.
name	Text	Product name, e.g. “Detergent 1kg”.
unit_price	Float	Selling price per unit.
unit_space_consumption	Float	Train-cargo space consumed per unit, e.g. 0.5.

Order entity

Property	Type	Description
id	Text	Primary key.
customer_id	Text	The customer who placed the order. (FK→Customer)
order_date	Timestamp	Date and time the order was placed.
requested_delivery_date	Date	Target delivery date, used to enforce the ≥ 7-day lead-time rule.
delivery_address	Text	Free-text delivery address.
route_id	Text	Route matched to the delivery address. (FK→Route)
status	Text	“Pending”, “Scheduled” or “Delivered”.

OrderItem entity

Property	Type	Description
id	Text	Primary key.
order_id	Text	The order this line belongs to.(FK→Order)
product_id	Text (FK→Product)	The product ordered. (FK→Product)
quantity	Integer	Quantity of this product on this order.

An order can contain multiple products; each product line on an order is stored as a separate OrderItem row.

TrainTrip entity

Property	Type	Description
id	Text	Primary key.
from_city	Text	Origin - always Kandy.
to_city	Text	Destination city.
trip_date	Date	Date of the trip.
departure_time	Timestamp	Scheduled departure.
arrival_time	Timestamp	Scheduled arrival.
max_capacity	Integer	Total cargo-space units available on this trip.

OrderTrainAllocation entity

Property	Type	Description
id	Text	Primary key.
order_item_id	Text	The order line being shipped. (FK→OrderItem)
train_trip_id	Text	The trip carrying (some or all of) this order line. (FK→TrainTrip)
quantity_allocated	Integer	How much of order_item_id's quantity rides on this specific trip.

Links order lines to train trips. When an order line exceeds a trip's remaining capacity, the remaining quantity is allocated to the next available trip via an additional row here.

Truck entity

Property	Type	Description
id	Text	Primary key.
vehicle_no	Text	License/registration number.
capacity	Float	Cargo capacity.

TruckTrip entity

Property	Type	Description
id	Text	Primary key.
truck_id	Text	Truck assigned. (FK→Truck)
driver_id	Text	Driver assigned. (FK→Worker)
assistant_id	Text	Assistant assigned. (FK→Worker)
route_id	Text	Route being run. (FK→Route)
start_time	Timestamp	Scheduled/actual start.

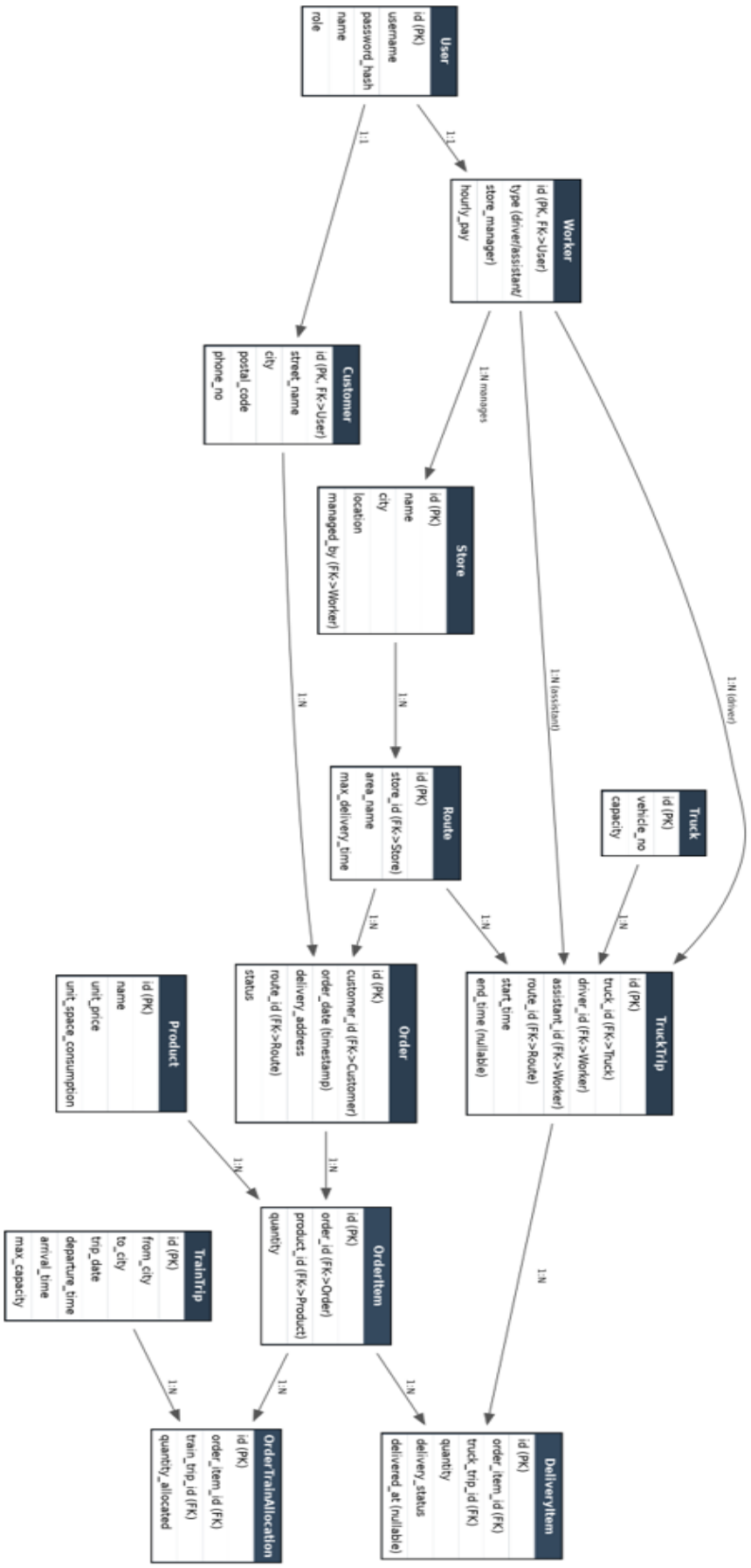
end_time	Timestamp, nullable	Null until the trip is completed.

DeliveryItem entity

Property	Type	Description
id	Text	Primary key.
order_item_id	Text	The order line being delivered. (FK→OrderItem)
truck_trip_id	Text	The truck trip carrying this order line on its last mile. (FK→TruckTrip)
quantity	Integer	Quantity of this order line on this truck trip.
delivery_status	Text	“Pending”, “Scheduled”, “Delivered”, or “Failed/Returned”.
delivered_at	Timestamp, nullable	Set when delivery_status becomes “Delivered.”

Tracks last-mile delivery status per order line, since different lines on the same order can go out on different truck trips.

ER Diagram


User
id (PK)
username
password_hash
name
role
Worker
id (PK, FK->User)
type (driver/assistant/store_manager)
hourly_pay
Customer
id (PK, FK->User)
street_name
city
postal_code
phone_no
Store
id (PK)
name
city
location
managed_by (FK->Worker)
Route
id (PK)
store_id (FK->Store)
area_name
max_delivery_time
Truck
id (PK)
vehicle_no
capacity
TruckTrip
id (PK)
truck_id (FK->Truck)
driver_id (FK->Worker)
assistant_id (FK->Worker)
route_id (FK->Route)
start_time
end_time (nullable)
Order
id (PK)
customer_id (FK->Customer)
order_date (timestamp)
delivery_address
route_id (FK->Route)
status
Product
id (PK)
name
unit_price
unit_space_consumption
OrderItem
id (PK)
order_id (FK->Order)
product_id (FK->Product)
quantity
TrainTrip
id (PK)
from_city
to_city
trip_date
departure_time
arrival_time
max_capacity
DeliveryItem
id (PK)
order_item_id (FK)
truck_trip_id (FK)
quantity
delivery_status
delivered_at (nullable)
OrderTrainAllocation
id (PK)
order_item_id (FK)
train_trip_id (FK)
quantity_allocated
1:1
1:N manages
1:N (assistant)
1:N (driver)
1:N

5.3 Stored Procedures and Functions

The following are implemented as PL/pgSQL procedures or functions, so that the rules they enforce cannot be bypassed by a direct or careless database write.

Procedure / Function	Purpose
place_order(customer_id, delivery_address, requested_delivery_date, items[])	Validates the seven-day advance rule and route coverage, then inserts the Order and all of its OrderItem rows as a single transaction. Either the whole order is saved or none of it is.
assign_order_to_train(order_item_id, trip_id)	Checks the trip's remaining cargo capacity against the order item's required space (quantity × unit_space_consumption) before inserting an OrderTrainAllocation row. If the current trip cannot take the full quantity, it allocates what fits and automatically looks for the next available trip on the same route for the remainder.
assign_truck_trip(truck_id, driver_id, assistant_id, route_id, start_time, end_time)	Checks for overlapping bookings and roster-limit breaches (weekly hours, consecutive assignments) before creating the TruckTrip row.
calculate_weekly_hours(worker_id, week_start)	Totals a worker's scheduled hours from TruckTrip for a given ISO week. Used by both the rostering trigger (Section 5.4) and the driver/assistant working-hours report.
update_delivery_status(delivery_item_id, new_status)	Moves a DeliveryItem through Pending → Scheduled → Delivered (or Failed/Returned), stamping the time of the change, and rolls the parent Order's status up once every DeliveryItem on it reaches a terminal state.

5.4 Triggers

- A BEFORE INSERT/UPDATE trigger on OrderTrainAllocation that blocks any allocation which would push a TrainTrip's total allocated space over its max_capacity.
- A BEFORE INSERT trigger on TruckTrip that blocks any truck, driver, or assistant being booked for an overlapping time window.
- A BEFORE INSERT trigger on TruckTrip that calls calculate_weekly_hours() and rejects the assignment if it would take the driver over 40 hours, or the assistant over 60 hours, in that week.
- A BEFORE INSERT trigger on TruckTrip that rejects a driver's third consecutive delivery, and an assistant's third consecutive route, in line with the roster rules.
- An AFTER UPDATE trigger on Order that records a timestamped status-history row whenever status changes, so delivery-performance reports have a full audit trail.
- An AFTER UPDATE trigger on DeliveryItem that checks whether all DeliveryItem rows for the parent Order are now “Delivered” and if so updates Order.status to “Delivered” automatically.

5.5 Indexing Strategy

Indexes are added where queries are expected to be frequent or where reports would otherwise need a full table scan.

- Foreign key columns are indexed to speed up joins:
  - Order.customer_id
  - Order.route_id
  - OrderItem.order_id
  - OrderItem.product_id
  - OrderTrainAllocation.order_item_id
  - OrderTrainAllocation.train_trip_id
  - TruckTrip.truck_id
  - TruckTrip.driver_id
  - TruckTrip.assistant_id
  - TruckTrip.route_id
  - DeliveryItem.order_item_id
  - DeliveryItem.truck_trip_id
- Order.order_date is indexed to support quarterly and date-range reports.
- Customer.city and TrainTrip.to_city are indexed to support the city-wise and route-wise sales breakdowns.
- TruckTrip.start_time is indexed to support monthly truck-usage analysis and to make the overlap-checking trigger efficient.
- A composite index on TruckTrip(driver_id, start_time) and TruckTrip(assistant_id, start_time) supports the weekly-hours calculation used by both the rostering trigger and the working-hours report.
- A composite index on TrainTrip(from_city, to_city, trip_date) supports locating the next available trip on a route when handling overflow.

5.6 Test Data

To demonstrate the schema and its constraints, the delivered database is populated with a realistic test dataset, in line with the project brief:

- At least 40 customer orders, spread across several months so quarterly reports have meaningful data.
- At least 10 delivery routes, covering the six destination cities named in the brief.
- A full train schedule with defined capacities, sufficient to exercise the overflow-to-next-trip logic.
- Enough truck trips, drivers and assistants to exercise the double-booking and working-hour triggers, including at least one case per trigger that is expected to be rejected, to prove the constraint works.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- Common user interactions (viewing orders, submitting a new order, viewing a schedule) should return within 1-2 seconds under normal load - realistic for a round trip that includes validation and a database write. A 100 ms target is not achievable for these interactions and is not set as a requirement.
- Reports covering a full quarter of data should complete within a few seconds, aided by the indexing described in Section 5.5.

6.2 Security Requirements

- Access is controlled by username and password login, with passwords stored as salted hashes rather than in plain text.
- Role-based access control ensures customers can only see and manage their own orders, while staff see only the functions relevant to their role.
- After logging in, a user receives a session token (e.g. a JSON Web Token) valid for 24 hours, so they are not asked to log in again during that period.
- All database writes that affect scheduling or working hours go through the procedures and triggers in Section 5, so the rules cannot be bypassed by direct data entry.

6.3 Software Quality Attributes

- Usability: interfaces for customers and staff should be simple enough to use with little to no training.
- Availability: the production system should target 99% uptime, consistent with the assumption in Section 2.7 that it runs on a persistent server.
- Maintainability: the data model and constraints are documented here and in the schema itself, so the rules governing scheduling and rostering are easy to find and change if the business rules change.
- Data integrity: enforced primarily at the database layer, so it holds regardless of which part of the application writes the data.

6.4 Business Rules

- An order must be placed at least seven days before its requested delivery date.
- An order can only be accepted if its delivery address is covered by an existing route.
- A train trip cannot be loaded beyond its stated cargo capacity; any overflow moves automatically to the next available trip.
- A truck, driver, or assistant cannot be booked for two overlapping trips.
- A driver must not be scheduled for two consecutive truck deliveries.
- An assistant must not be scheduled for more than two consecutive routes.
- A driver cannot work more than 40 hours in a week; an assistant cannot work more than 60 hours in a week.

7. Other Requirements

For now, only English is supported throughout the system. Support for other languages may be added later.

Appendix A: Glossary

Terms

Term	Definition
Admin	The user who manages master data (routes, products, train schedules) and generates management reports.
Assistant	A staff member who helps the driver during last-mile delivery.
Cargo Capacity	The total space available on a train trip; a hard limit when scheduling orders.
Customer	A user who places orders for delivery to a given address.
Driver	A staff member who drives the truck used for last-mile delivery.
Order	A customer request for one or more products, with quantities and a delivery address.
Order Item	A single product line within an order, holding the product and its quantity.
Order Status	The current stage of an order: Pending, Scheduled, or Delivered.
Product	A physical item in Kandypack's catalogue, with a unit price and a space-consumption value.
Route	A predefined delivery path with an associated maximum delivery time.
Store	The depot near a railway station where goods are unloaded before last-mile delivery.
Train Trip	A scheduled journey carrying cargo between Kandy and a destination city.
Truck Trip	A single scheduled journey of a truck, with an assigned driver, assistant and route.
Working Hours	The weekly hour limit for staff: 40 hours for drivers, 60 hours for assistants.

Acronyms

Acronym	Definition
SCDS	Rail and Road-based Supply Chain Distribution System.
FMCG	Fast-Moving Consumer Goods.
API	Application Programming Interface.
SQL	Structured Query Language.
REST	Representational State Transfer (for web services).
JSON	JavaScript Object Notation (for data exchange).
JWT	JSON Web Token (for user authentication).
UI	User Interface.
DBMS	Database Management System.

Abbreviations

Abbreviation	Definition
KPI	Key Performance Indicator.
SLA	Service Level Agreement.

Appendix B: Analysis Models

Entity-relationship diagrams and any further analysis models will be added as a separate diagram once the physical schema is finalised; the entity list in Section 5.2 is the current source of truth for the data model.

Appendix C: To Be Determined List

- Exact user interface designs and visual guidelines will be decided during implementation.
- Final product dimensions and space-consumption figures will be confirmed against real Kandypack catalogue data where available.